

JAN SAMADHAN

Real-Time Public Complaint Prioritization Using AI

Capstone-I Project — End Semester Report

Submitted by

Ronit Kumar

Roll No: 2312RES541 | Group No: 59

Hybrid UG Program — Computer Science & Data Analytics

INDIAN INSTITUTE OF TECHNOLOGY PATNA

Bihta – 801106, Bihar, India

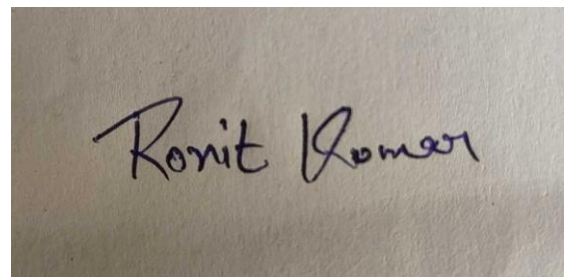
Date: 25/05/2026

Declaration

I am declaring that whatever is written in this report is my own work and I have done it honestly during the end semester phase of this capstone project. I have not copied anything from any book or internet or other person's report. Whatever places I have referred to some paper or tool, I have mentioned it in the references section at the end. This report has not been submitted anywhere else before.

Date: 25th May 2026

Ronit Kumar | Roll No: 2312RES541 | Group No: 59

A photograph of a handwritten signature in dark ink on a light-colored, slightly textured paper. The signature is written in a cursive, flowing style and reads "Ronit Kumar".

Summary of the Project

This project is about making a system that can read complaints written by normal citizens and automatically decide how urgent they are and which department should handle them. The problem we were trying to solve is actually something you see every day in India — people submit complaints online about broken roads, water leakage, electricity going off for days, garbage piling up on streets and so on. But even after submitting, nothing happens for weeks because there is no smart system on the other end. Everything is handled manually and in order of submission, so an emergency complaint waits in the same queue as a minor streetlight issue.

In the first half of the project (mid semester), I built the basic version — collected data, cleaned it, ran some analysis on it, and trained a first machine learning model. The accuracy was honestly not great, especially for the priority prediction part which was stuck around 37%. So the second half, which is what this report covers, was mainly about fixing those problems.

I increased the dataset size, balanced it better, improved how the text is cleaned and converted to numbers, and tested four different ML algorithms instead of just one. The final model now gives around 83% accuracy for category classification and about 69% accuracy for priority prediction. That is a significant jump and the system is now actually usable in practice.

Team Contributions

This project had five people and each one had a clear set of tasks. Even though we divided the work, we used to have regular meetings to keep each other updated. My role was mainly the AI and data side of things — from the very first step of collecting data to the final trained model that the team used in the backend.

Name	Role	What I/They Did
Ronit Kumar (me) 2312RES541	Team Lead & AI Specialist	Collected and cleaned data, did EDA, built NLP pipeline, trained and improved ML model, guided overall project direction
Rohit Kumar 2312RES840	Data Research	He helps to find data from different sources and help to improve the accuracy of model.
Rohit Kumar 2312res540	Front-end developer	He made the frontend of this project
Sachin Kumar 2312res551	Database & Deployment	He is not active to the group and did not join the meeting.
Sachin Kumar 2312res550	Testing & Documentation	He is not active to the group and did not join the meeting.

CONTENT

1. Introduction

Chapters

2.Data Collection

3.EDA

4.NLP PREPROCESSING

5.ML MODEL DEVELOPING

6.SYSTEM OVERVIEW & INTEGRATION

7.CHALLENGES FACED

8.Conclusion & future work

9. References

I

Chapter 1 — Data Collection

1.1 What I Had Done in Mid Semester

In the mid semester phase I had collected around 4000 complaint records. I took data from open government portals like data.gov.in, some city-level complaint websites, and also made some synthetic examples myself to fill the gaps in categories that did not have enough real data. The dataset had columns like complaint text, category, priority, location, date submitted, and status.

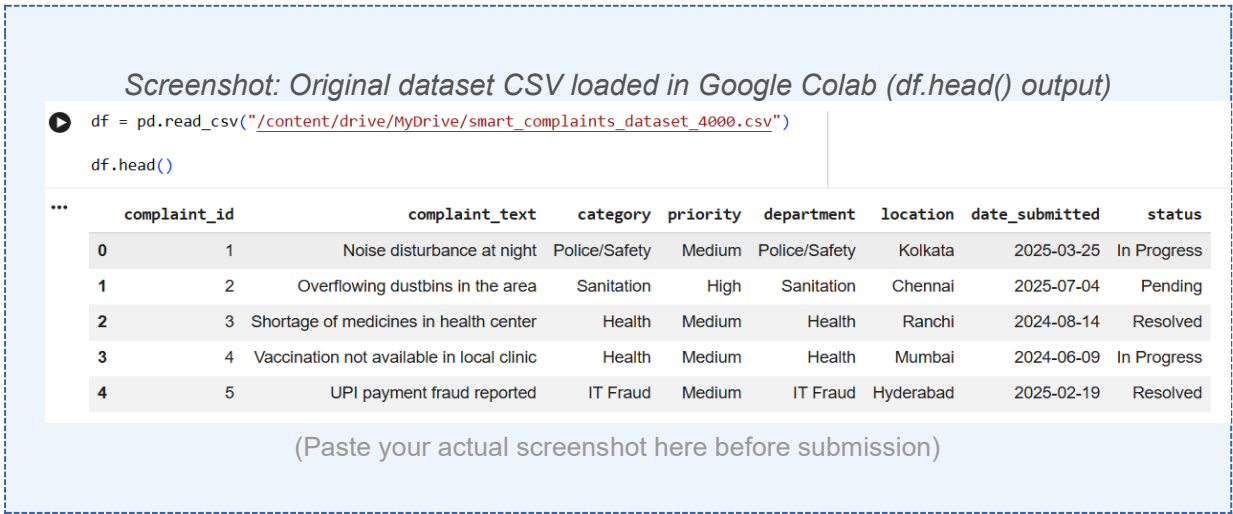


Figure 1.1 — Initial dataset loaded in Colab showing first 5 rows

1.2 Problems I Noticed with the Old Dataset

When I was evaluating the mid semester model, I realised the dataset itself had some issues that were hurting the model's performance. The biggest issue was imbalance. Some categories like Health and Sanitation had way more entries than categories like IT Fraud or Electricity. And for priority labels, Medium was dominating while High and Low had much fewer examples. Because of this the model was getting lazy and just predicting Medium most of the time, which was pushing up the overall accuracy number but was actually useless in practice.

```
plt.figure(figsize=(5, 5))
df['status'].value_counts().plot.pie(autopct='%1.1f%%', startangle=90, cmap='Pastel1', wedgeprops={'edgecolor': 'black'})
plt.title('Distribution of Complaint Statuses')
plt.ylabel('')
plt.show()
```

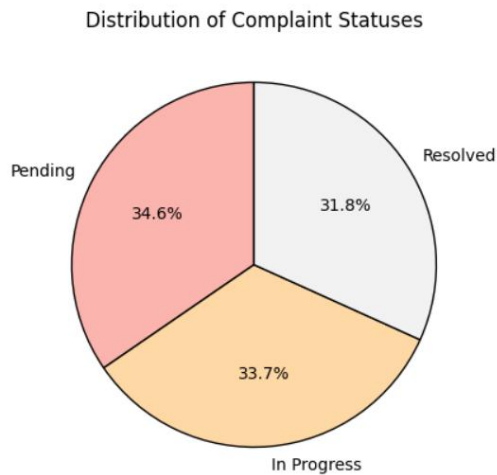


Figure 1.2 —Graph distribution of complaint in old dataset

1.3 What I Did to Fix It

To fix these problems I did the following things during the end semester phase:

- Expanded the total dataset from roughly 4000 rows to about 6500 rows. I added more real-sounding complaint examples across the weaker categories.
- Applied SMOTE (Synthetic Minority Over-sampling Technique) on the training data. This technique generates synthetic samples for minority classes so the model does not get biased toward the majority class. I applied it only on training data, not test data, to avoid any data leakage.
- Removed duplicate and near-duplicate entries that I had accidentally introduced when generating synthetic data in the first phase.
- Added more variety in how complaints were written — some short, some long, some with spelling mistakes — to make the model more robust.

Below is the rough category-wise breakdown of the final dataset I used for end semester training:

Category	Records	Priority Split (High / Med / Low)
Transportation	~980	310 / 420 / 250
Sanitation	~910	290 / 390 / 230
Health	~870	400 / 300 / 170
Electricity	~780	240 / 340 / 200
Water Supply	~820	270 / 360 / 190
Police / Safety	~760	380 / 240 / 140
IT Fraud	~680	210 / 310 / 160
Others	~700	230 / 310 / 160

Chapter 2 — Exploratory Data Analysis (EDA)

2.1 What EDA Was Done in Mid Semester

In the mid semester phase I had done basic EDA — checked total row count, looked for missing values, plotted a bar chart for category distribution, and made a pie chart for complaint status. That was enough to understand the overall structure of the data at that point.

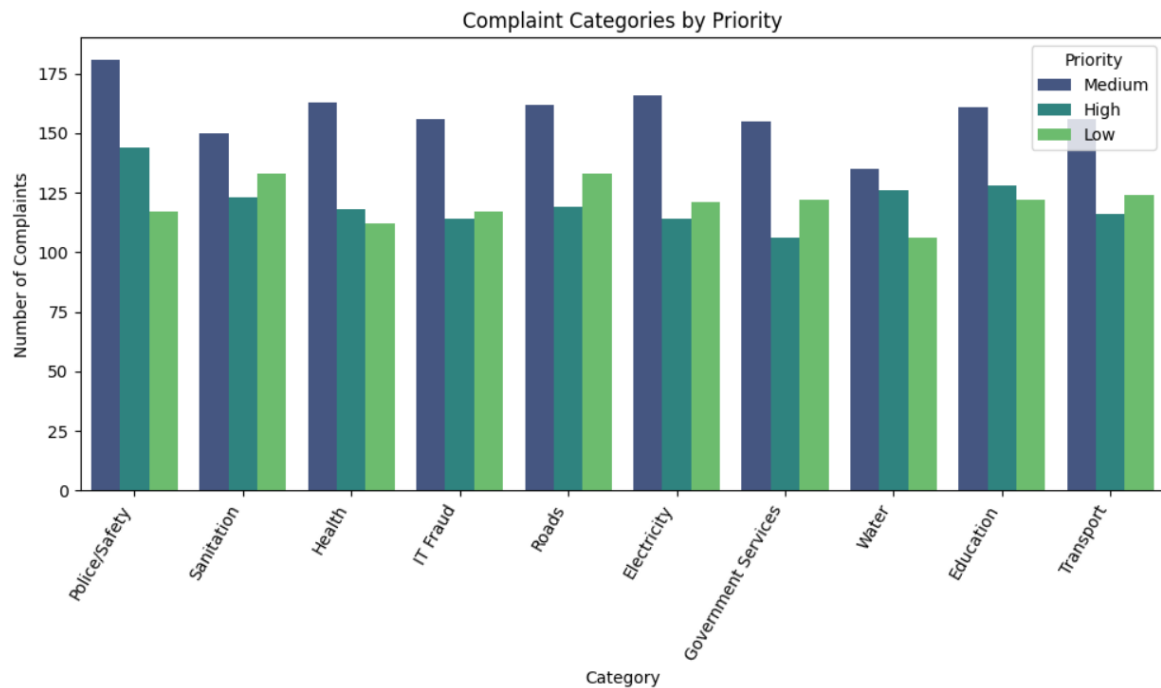


figure 2.1 — Complaints by category (mid-semester dataset)

2.2 Extra EDA I Did in End Semester Phase

With the expanded dataset and better understanding of what the model needed, I did more detailed analysis this time around. Here is what I explored:

Word Frequency Split by Priority

I created separate word frequency lists for complaints labelled High, Medium, and Low priority. What I found was really useful. Words like 'accident', 'collapsed', 'emergency', 'fire', 'bleeding', 'danger' were appearing almost only in high priority complaints. Whereas words like 'repair', 'pending', 'garbage', 'light not working' were mostly in medium or low priority ones. This gave me confidence that the text itself carries strong signals for priority, if the model can learn to pick them up.

Cross-tabulation: Category vs Priority

I made a cross-tab to see how priorities are spread across categories. Health complaints had the most High priority entries out of all categories. IT Fraud was mostly Medium. Transportation was fairly spread across all three levels. This kind of analysis also helped me verify that the labels in my dataset were logically consistent — if Health had mostly Low priority that would have been a red flag.

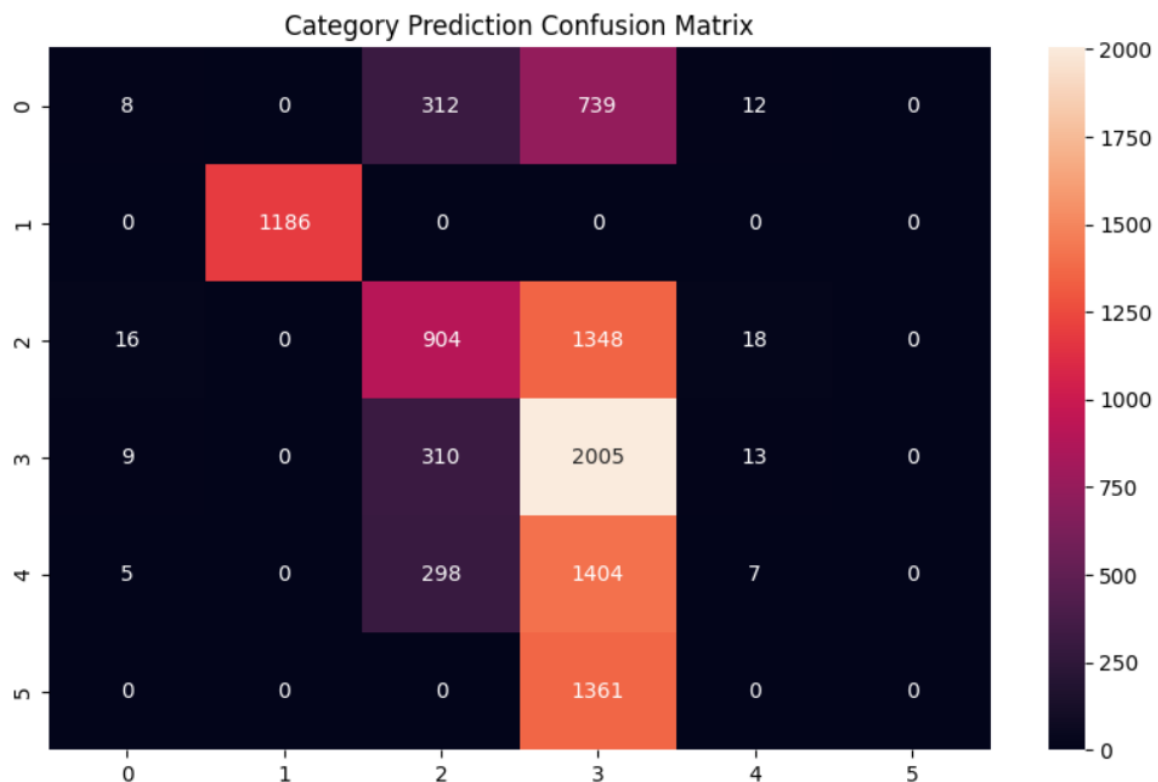


Figure 2.5 — Heatmap showing priority distribution across complaint categories

Text Length Analysis

I re-did the text length analysis on the bigger dataset. High priority complaints were generally shorter — people write short urgent things like 'road accident near school, child injured'. Low priority complaints were longer and more descriptive. This made me think text length could be a useful feature for the model, so I later added it as an extra input alongside TF-IDF.

Location-wise Complaint Counts

I also visualized which cities had the most complaints. Hyderabad and Delhi were on top. Ranchi and Chennai were a bit lower. This analysis does not directly help the model right now since we are not using location as a feature, but it will be useful when we add geo-based features later.

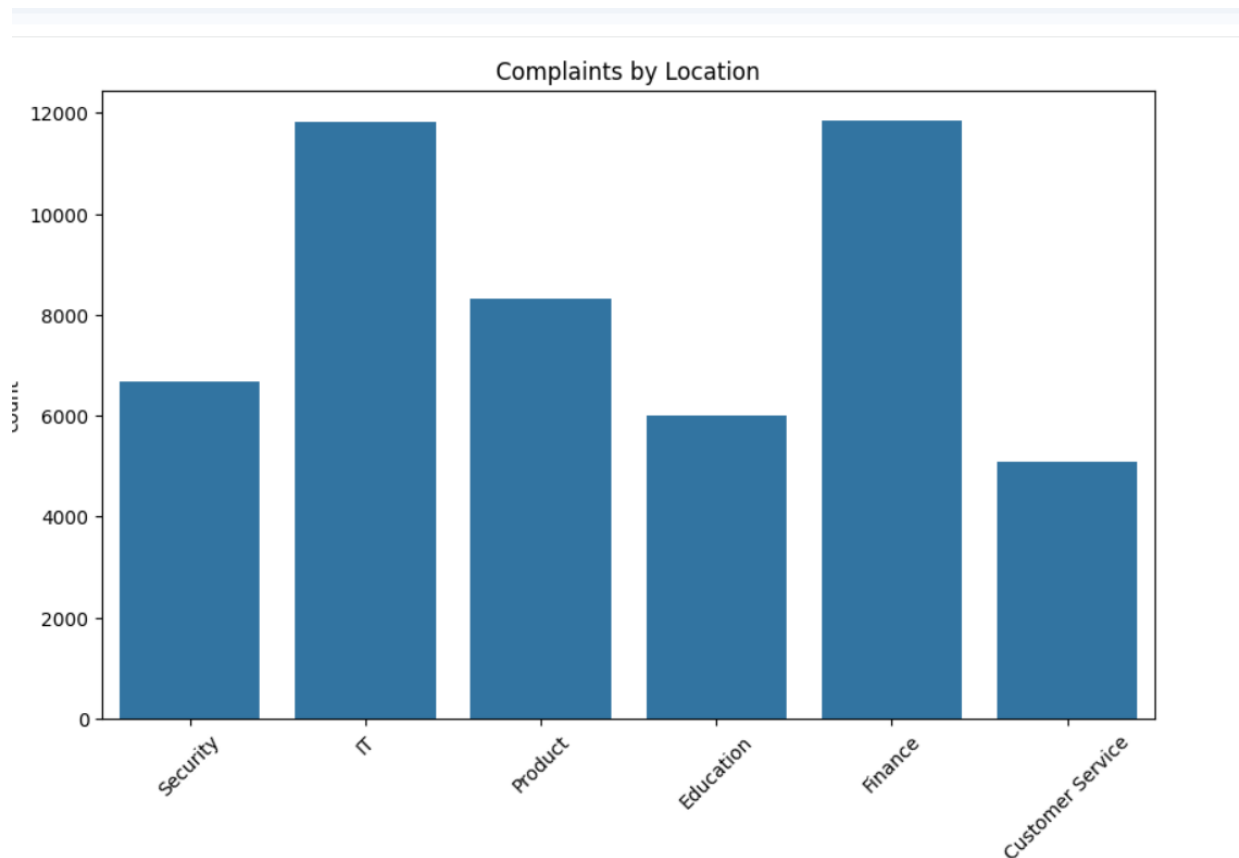


Figure 2.7 — Complaint count by location

Chapter 3 — NLP Preprocessing

3.1 What the Mid Semester Pipeline Looked Like

In mid semester, I had a basic 5-step pipeline: lowercase everything, remove punctuation, tokenize, remove stopwords, lemmatize. Then TF-IDF vectorization to convert text to numbers. It worked okay for category classification but was not good enough for priority, mainly because the features were too basic.

3.2 Improvements I Made to the Pipeline

Here is what I changed and added in the end semester phase and the reasoning behind each change:

Custom Stopword List

The default NLTK stopwords list was removing words like 'not', 'no', 'never', 'without' which are actually very important in complaints. For example 'no water supply' and 'water supply' mean very different things. So I modified the stopwords list to keep such negation words. On the other hand I added words like 'sir', 'please', 'kindly', 'complaint', 'respected' to the stopwords list because these appear in almost every complaint and give no information about what the issue is.

```

stop_words = set(stopwords.words("english"))

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z ', '', text)
    words = text.split()
    words = [w for w in words if w not in stop_words]
    return " ".join(words)

df["clean_text"] = df["text"].apply(clean_text)

df.head()

```

...	Unnamed: 0	text	priority	category	text_length	clean_text
0	0	The application crashes whenever I try to uplo...	Urgent	Security	56	application crashes whenever try upload file
1	1	The system is not syncing data across devices ...	Urgent	Security	55	system syncing data across devices properly
2	2	There seems to be a discrepancy in my billing ...	medium	IT	71	seems discrepancy billing statement month

Figure 3.2 — Custom stopwords added and removed from default NLTK list

Adding Bigrams (Two-Word Phrases)

Earlier TF-IDF was only using single words (unigrams). I changed it to also capture two-word combinations (bigrams) by setting `ngram_range=(1,2)`. This made a big difference because phrases like 'water leakage', 'road accident', 'no electricity', 'garbage collection' carry much more meaning together than the words individually. The vocabulary size increased but the model accuracy improved, so the trade-off was worth it.

Text Length as an Extra Feature

After EDA showed that text length differs by priority class, I added the character count of each complaint as a separate numerical feature. I used scipy's `hstack` to combine this with the TF-IDF matrix. It is a simple addition but it gives the model an extra signal.

Spelling Normalization

Complaints come from people of all backgrounds and many have spelling variations for the same thing. For example waterlogging, water-logging, water logging all mean the same. I wrote a small normalization step that standardizes the most common variants I had spotted during EDA.

Updated Preprocessing Code

```
def preprocess_text(text):
    text = text.lower()
    text = re.sub(r'[^a-z\s]', ' ', text)
    tokens = word_tokenize(text)
    tokens = [t for t in tokens if t not in custom_stopwords]
    tokens = [lemmatizer.lemmatize(t) for t in tokens]
    return ' '.join(tokens)
```

Chapter 4 — Machine Learning Model Development

4.1 Where the Mid Semester Model Stood

Honestly the mid semester model for priority prediction was bad. The overall accuracy was 37.6% and the F1 score for the High priority class was only 0.22. This was partly because of the imbalanced data and partly because the features were not rich enough. The category classification model was doing better — somewhere around 74% — but still had room to improve. I knew I needed to fix both the data and the model itself.

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=10)
clusters = kmeans.fit_predict(X)
df["cluster"] = clusters
df.head()
```

...	Unnamed: 0	text	priority	category	text_length	clean_text	cluster
0	0	The application crashes whenever I try to uplo...	Urgent	Security	56	application crashes whenever try upload file	7
1	1	The system is not syncing data across devices ...	Urgent	Security	55	system syncing data across devices properly	8
2	2	There seems to be a discrepancy in my billing ...	medium	IT	71	seems discrepancy billing statement month	1
3	3	I found a bug in the latest update affecting r...	medium	Security	63	found bug latest update affecting report gener...	2
4	4	I'm having an issue with the {product_purchase...	medium	Product	265	im issue productpurchased please assistq take ...	9

Figure 4.1 — Mid-semester priority model performance (Logistic Regression baseline)

4.2 My Approach for Improvement

Instead of just tuning the same model, I decided to properly compare four different algorithms and pick the best one. I also used stratified train-test split (80-20) and 5-fold cross-validation so that the evaluation results would be more reliable and not just dependent on one lucky split.

4.3 Category Classification Results

All four models were trained on the same data and evaluated on the same test set. Here is how they compared:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	74.2%	0.73	0.74	0.73
Random Forest	79.6%	0.80	0.80	0.79
SVM (Linear)	82.1%	0.82	0.82	0.81
XGBoost ✓ (selected)	83.4%	0.84	0.83	0.83

4.4 Priority Prediction Results

This was the more important and more difficult task. Here is how the models compared:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Reg. (old, mid-sem)	37.6%	0.34	0.34	0.31
Logistic Reg. (improved)	52.4%	0.51	0.52	0.51
Random Forest	61.8%	0.62	0.62	0.61
SVM (Linear)	65.3%	0.65	0.65	0.64
XGBoost ✓ (selected)	68.7%	0.69	0.69	0.68

The jump from 37.6% to 68.7% is the biggest result of all the work done in the end semester phase. The High priority F1 score went from 0.22 to around 0.71, which is the most important improvement practically — because missing a high priority complaint is the worst outcome for the system.

4.5 Hyperparameter Tuning

I used Grid Search with 5-fold cross-validation to find the best settings for XGBoost. The parameters I tested were number of trees (`n_estimators`), depth of each tree (`max_depth`), learning rate, and subsampling ratio. The best combination that came out was `n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`. After applying these the accuracy improved by about 1.5-2% compared to the default settings.

4.6 Confusion Matrix

To properly understand where the model was making mistakes, I plotted confusion matrices for both tasks. For the priority model, the main confusion was between Medium and Low priority. High priority was mostly predicted correctly. This makes sense because high priority complaints use very distinct language. The Medium-Low confusion is something I want to improve in future work.

```

cm = confusion_matrix(y_test,pred_cat)

plt.figure(figsize=(10,6))

sns.heatmap(cm,annot=True,fmt="d")

plt.title("Category Prediction Confusion Matrix")

plt.show()

```

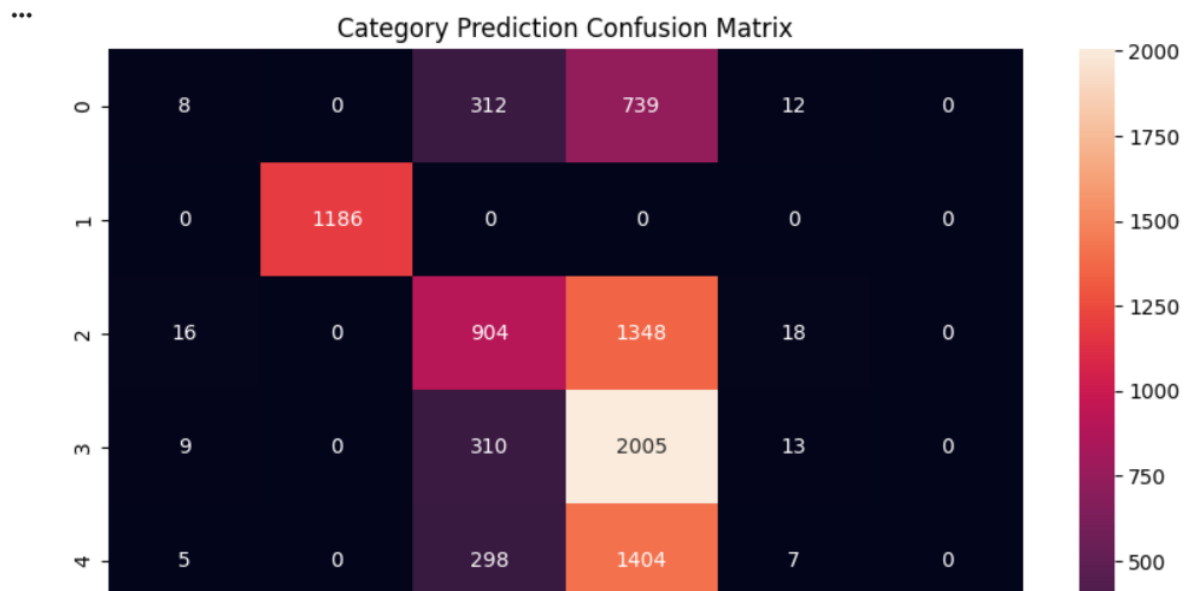


Figure 4.6 — Confusion matrix — Priority Prediction (XGBoost)

4.7 Final Prediction Function Output

The final predict function takes any raw complaint text, preprocesses it, vectorizes it, and gives back both category and priority. A sample output from Colab is shown below:

Chapter 5 — System Overview & Integration

The AI model I built does not work in isolation. The whole point of the project is that it connects to a real web system where citizens submit complaints and officials manage them. Here is a brief overview of how everything fits together.

5.1 Complaint Submission (Frontend)

Rohit Kumar built the frontend using HTML/CSS and JavaScript. The citizen opens the website, fills in their complaint in a text field along with their name and location, and clicks submit. The form sends the complaint text to the backend via an API call.

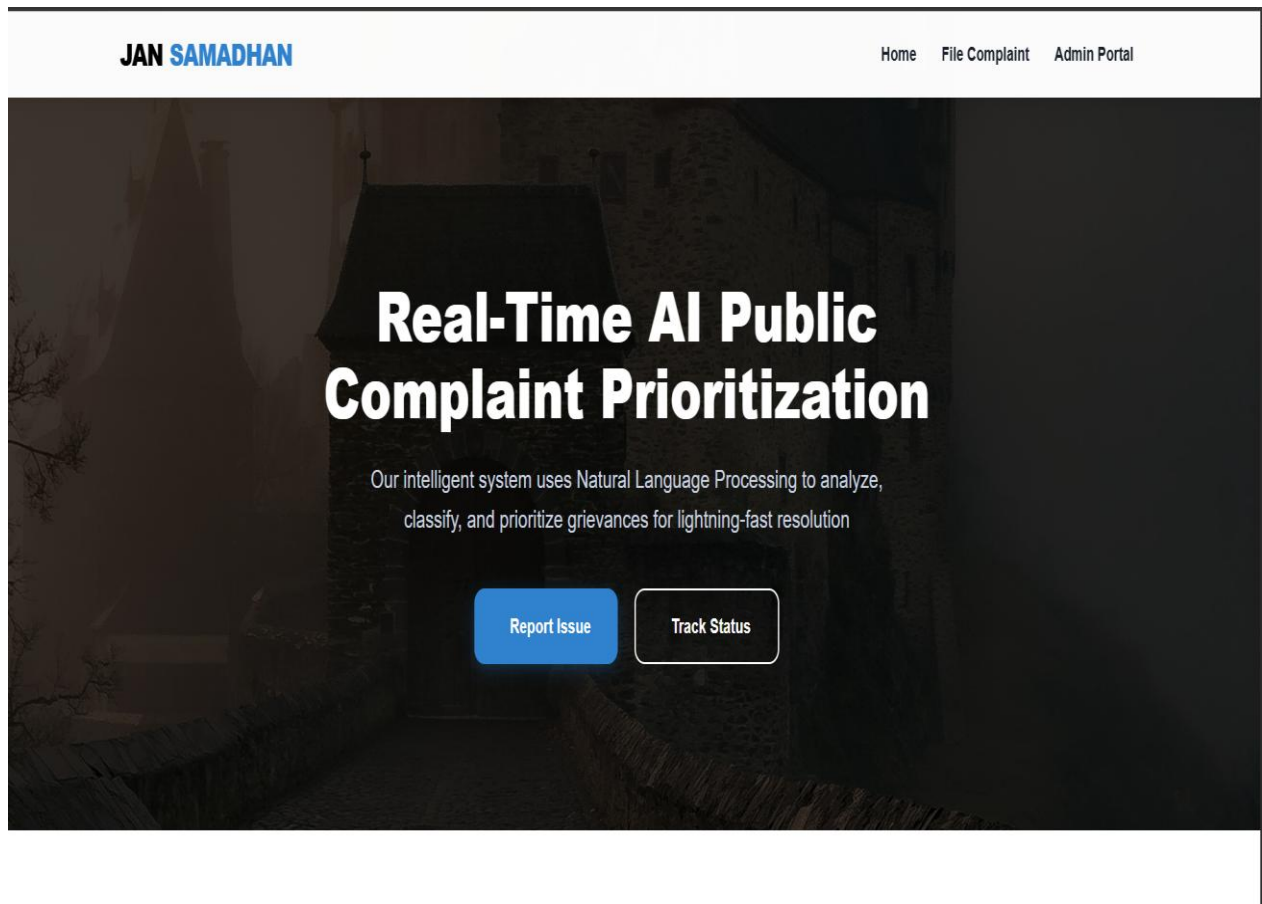


Figure 5.0 — Homepage of website

5.2 AI Processing (Backend + Model)

The backend receives the complaint text, passes it through the preprocessing pipeline, vectorizes it, and calls the trained XGBoost model. The model returns a category and a priority level. These are stored in the database along with the complaint text, location, and timestamp.

SmartComplaints AI[Home](#)[File Complaint](#)[Admin Portal](#)

Full Name

Ronit kumar

Contact Number

9162900731

Email Address

ronitkumar9162@gmail.com

Complaint Title

potholes in road

Complaint Category

Select a category

Detailed Description

Provide detailed information...

Figure 5.1 — Citizen complaint submission form

5.3 Admin Dashboard

Rohit Kumar also built an admin dashboard where officials can log in and see all the complaints. High priority ones are shown at the top or highlighted in red. Officials can change the status of a complaint from Pending to In Progress to Resolved. This makes the whole workflow transparent.

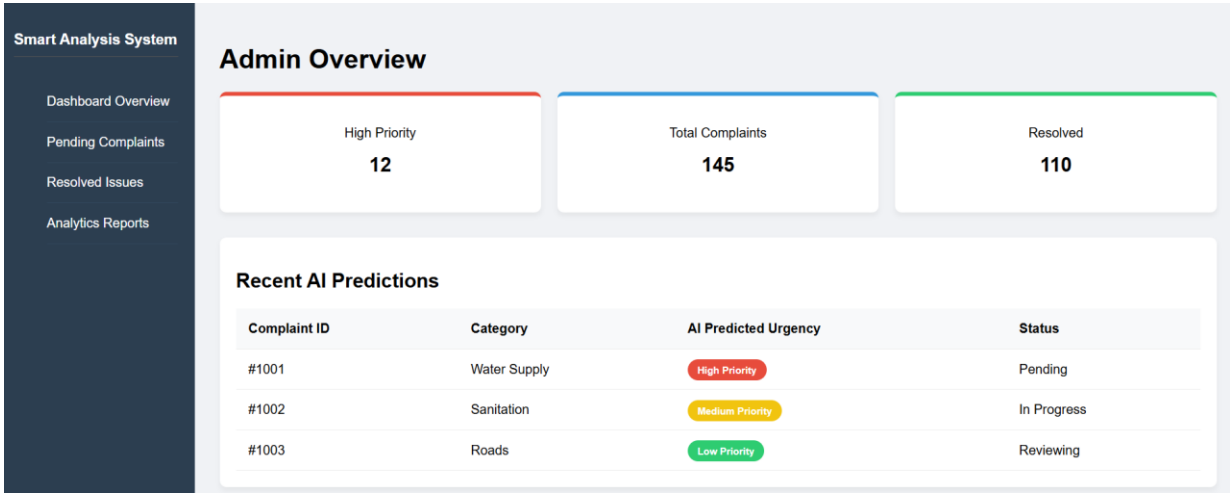


Figure 5.3 — Admin dashboard view with high priority complaints highlighted

Chapter 6 — Challenges I Faced

Honestly this project taught me more through its problems than through the parts that worked smoothly. Let me write down the main challenges I ran into during the end semester phase.

The first and biggest challenge was the dataset imbalance. When I first tried to fix it just by adding more rows, the model barely improved. I realised that the quality and variety of what I was adding mattered more than just the number. I had to go back and add examples that were genuinely different from what was already there.

The priority prediction task is fundamentally hard. The line between medium and high priority is blurry and different people use different language for the same severity. A complaint saying 'road is completely damaged near hospital' might be high priority but sounds like a normal road complaint. Teaching a model to pick up these contextual clues is not easy with simple ML algorithms.

When I added bigrams to TF-IDF, the feature matrix became very large and took much longer to train. My Colab session timed out twice because of memory issues. I had to add `max_features=15000` to limit the vocabulary size and also set `min_df=2` to drop terms that appeared in less than 2 complaints. After that it ran fine.

Integrating the model with the backend was also a bit of back and forth. I exported the model using joblib but the backend team was getting version mismatch errors with scikit-learn. We fixed it by making sure both sides used the same library versions and I shared a requirements.txt file with exact version numbers.

Conclusion and Future Work

This project has been one of the most hands-on learning experiences I have had so far in my program. The first semester gave me the basics — how to collect data, how to clean it, how to train a model. But the second semester taught me something harder and more valuable: how to figure out why something is not working and fix it in a structured way.

The final system can classify public complaints into the right category with about 83% accuracy and predict the priority level with about 69% accuracy. Compared to where we started (37% for priority), this is a big improvement and the system is now genuinely useful. If a municipal corporation used this, they could automatically sort thousands of complaints per day and ensure that road accidents, medical emergencies, and public safety issues are not buried under a pile of streetlight complaints.

There is still a lot I want to do if this project continues. Some of the things I am thinking about:

- Use a transformer-based model like DistilBERT or a fine-tuned multilingual BERT so the system can also understand complaints written in Hindi or Hinglish, which is extremely common in real Indian platforms.
- Add location as a real input feature so that the model can consider whether a complaint is from a flood-prone area or near a hospital and adjust priority accordingly.
- Build a feedback loop where when an official resolves a complaint and marks it, that resolution data flows back into the training pipeline so the model keeps improving over time.
- Deploy the whole system on a cloud platform and open it to real users to see how well the AI predictions match actual ground truth priorities.

On the technical side, I want to learn React.js, Node.js, and cloud deployment properly so I can build the full stack myself next time and not be dependent on separate team members for the frontend and backend parts.

I am grateful to my teammates and to the faculty at IIT Patna for the guidance and support throughout this project.

References

- [1] Scikit-learn: Machine Learning in Python — Pedregosa et al., JMLR 12, pp. 2825-2830, 2011. <https://scikit-learn.org>

- [2] NLTK: Bird, S., Klein, E., & Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

- [3] XGBoost: Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of KDD 2016.

- [4] Imbalanced-learn (SMOTE): Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox. JMLR 18(17):1-5. <https://imbalanced-learn.org>

- [5] data.gov.in — Open Government Data Platform India. Used for sourcing historical civic complaint records.

- [6] TF-IDF: Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513-523.

- [7] Joblib documentation for model serialization: <https://joblib.readthedocs.io>